

Legal Chatbot Project Presentation & FAQs

1. How the Project is Made (Architecture & Workflow)

The Legal Chatbot is designed as a multilingual, voice-enabled RAG (Retrieval-Augmented Generation) application to help users in India understand their legal rights (KYR - Know Your Rights).

The workflow is structured into several interconnected modules:

1. User Interface (UI): Built using Streamlit, offering both text and voice input capabilities. Users can speak or type their queries in Hindi, Tamil, or English.
2. Voice Processing: If a user speaks, the SpeechRecognition library converts the audio to text. For responses, gTTS (Google Text-to-Speech) converts the final text answer back into an audio file.
3. Natural Language Processing (NLP) Engine:
 - Language parsing and entity extraction are handled by spaCy and Regex.
 - The user query is converted into dense vector embeddings using the `sentence-transformers` model (paraphrase-multilingual-MiniLM).
 - This embedding is compared against a pre-indexed knowledge base (FAQs) using Cosine Similarity to identify the user's intent.
4. Database & Knowledge Base (Legal DB): SQLite stores an extensive collection of Indian legal FAQs (derived from the Constitution, IPC, BNS, CrPC, etc.) in a structured JSON/SQLite format.
5. Generative AI (Gemini Integration): If the system cannot find a direct match in the database, or needs to synthesize a complex answer, it queries the Gemini API. The retrieved database context is injected into the Gemini prompt (RAG approach) to ensure the AI's response is legally accurate, localized, and context-aware.

2. Technologies & Libraries Used

- Frontend: Streamlit (Rapid web-app prototyping, mobile-first design UI components)
- Programming Language: Python 3
- NLP & Machine Learning:
 - * transformers & sentence-transformers: For multilingual intent recognition and text embeddings.
 - * spacy: For basic NLP, tokenization, and entity extraction.
 - * scikit-learn: For Cosine Similarity calculations to match queries with database intents.
 - * torch: Backend framework for the transformers model.
 - * nltk & indic-nlp-library: For extended language processing capabilities.
- Voice Processing:
 - * SpeechRecognition: Audio-to-text conversion.
 - * gTTS (Google Text-to-Speech): Text-to-audio generation.
- Database: SQLite (via Python's built-in sqlite3) for fast, local querying.
- Generative AI Integration: google-generativeai (Gemini API for the LLM brain).
- Environment Management: python-dotenv.
- Deployment: Ready for Streamlit Cloud or Vercel.

Legal Chatbot Project Presentation & FAQs

3. FAQs / External Examiner Questions

Q1: Why did you choose Streamlit instead of React/Django for this project?

Answer: Streamlit allows for rapid prototyping and seamless integration with Python's data science and NLP libraries. Since the core complexity of this project is the NLP engine, RAG pipeline, and AI integration, Streamlit was the most efficient way to build a responsive, functional UI without significant frontend overhead.

Q2: How does the Retrieval-Augmented Generation (RAG) pipeline work in your chatbot?

Answer: When a user asks a question, we first generate an embedding of their query using sentence-transformers. We use cosine similarity to find the most relevant intent in our local legal knowledge base (SQLite/JSON). This retrieved text acts as our "ground truth" context. We then send this context, along with the user's question, to the Gemini LLM. This constraints the AI to only answer based on Indian law, preventing hallucinations.

Q3: What happens if the Gemini API goes down or the user asks an out-of-scope question?

Answer: The system has fallbacks. If the API is down, we rely on the local vector search to provide the closest matching FAQ answer directly. For out-of-scope questions, our system prompt strictly instructs the LLM to politely state that it can only assist with legal queries.

Q4: How did you handle the multilingual aspect (Hindi, Tamil, English)?

Answer: We used the `paraphrase-multilingual-MiniLM-L12-v2` sentence transformer, which natively understands multiple languages and maps them to the same vector space. This means a query asked in Hindi will land close to the English or Tamil equivalent in our knowledge base, ensuring accurate intent matching regardless of the input language.

Q5: Why use sentence-transformers when Gemini itself is an LLM?

Answer: Cost, speed, and accuracy. By performing local intent matching first with sentence-transformers, we can instantly answer common FAQs without hitting the Gemini API, reducing latency and cost. Gemini is reserved for complex generation and synthesizing answers when a direct match isn't perfect.

Q6: What measures did you take to ensure the legal advice is accurate and not misleading?

Answer: The chatbot is strictly configured via prompt engineering to act as a legal assistant, not a lawyer. We inject explicit system instructions to only use the provided context from our verified database. Additionally, it is programmed to say "I need more info" or suggest consulting a lawyer for highly complex cases rather than guessing.